

42

Docker

Package your app to run the same anywhere

The standard way to ship real software

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A tool to package apps into containers
A container	Your app + everything it needs to run
The win	Runs identically on any machine
Fixes	'But it works on my computer!'
For you	A key skill-gap target for ML jobs

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	The Problem Docker Solves	'Works on my machine'
2	What is Docker?	Containers explained
3	What's in a Container?	Everything the app needs
4	Images vs Containers	Blueprint vs running copy
5	The Dockerfile	The recipe for your app
6	The Main Commands	Build, run, and more
7	Dockerizing an ML App	Your FastAPI model
8	Why It Matters for Jobs	A real-world skill
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. The Problem Docker Solves

Every developer has hit this: your app runs perfectly on your computer, but breaks when you put it on a server or send it to a teammate. Why? Different Python versions, missing libraries, different operating systems, different settings. The classic cry is 'but it works on my machine!'

Docker fixes "but it works on my machine!"

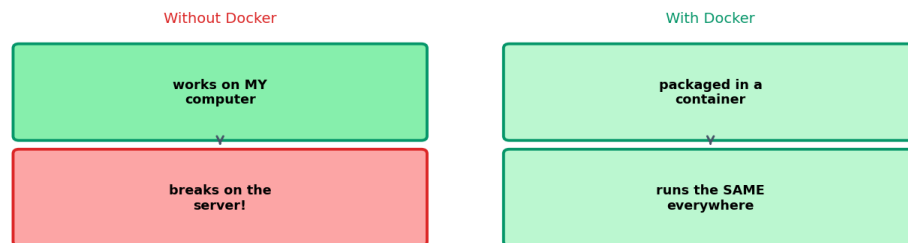


Fig 1 — Without Docker, code that works for you can break elsewhere. Docker makes it run the same everywhere.

Docker solves this completely. It packages your app together with everything it needs — the right Python version, all libraries, system tools, and settings — into one bundle called a container. That container then runs exactly the same way on any computer, server, or cloud. No more 'works on my machine' surprises.

■ *Docker is one of your named skill-gap targets, and for good reason — it's expected in almost every ML engineering role. Learning it bridges the gap between 'I trained a model' and 'I can ship it to production reliably'.*

2. What is Docker?

Docker is a tool that packages software into containers. A container is a lightweight, self-contained box that holds your app and everything it needs to run. Whatever's inside the container behaves identically no matter where the container runs.

A helpful comparison: think of a shipping container. Before standardised shipping containers, loading cargo was a mess — every item was a different shape. The container standardised everything: any container fits any ship, truck, or crane. Docker does this for software — your app, packaged in a standard container, runs anywhere that has Docker.

■ *Containers are lightweight — they share the host computer's core, so they start in seconds and don't waste resources. This is why Docker became the standard way to package and ship software, including ML models.*

3. What's in a Container?

A container holds everything your app needs to run, bundled together. That's what makes it portable — nothing is left to chance on the target machine.

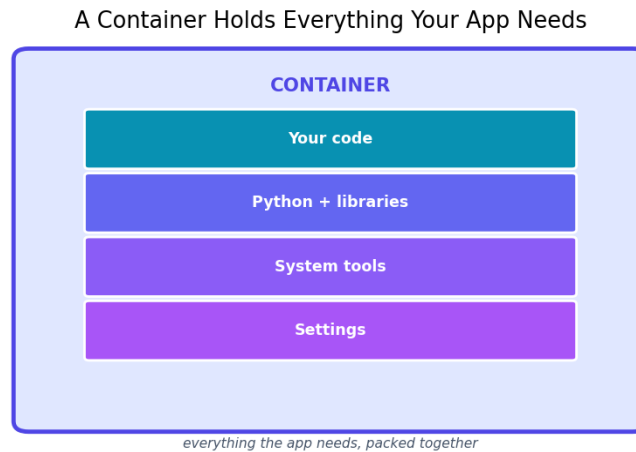


Fig 2 — A container packs your code, the right libraries, system tools, and settings — all together.

Inside a container	Example
Your code	Your FastAPI app, your model file
The runtime	The exact Python version you need
Libraries	torch, fastapi, numpy — exact versions
System tools	Anything the OS-level app needs
Settings	Environment variables, config

Because all of this travels together inside the container, there's no 'I forgot to install that library' or 'wrong Python version' on the server. The container is a complete, sealed package. Whatever ran in your container on your laptop runs the same in your container on the cloud.

4. Images vs Containers

Two words you'll always hear: image and container. They're closely related but different. An image is the blueprint — a saved template of your packaged app. A container is a running copy made from that image.

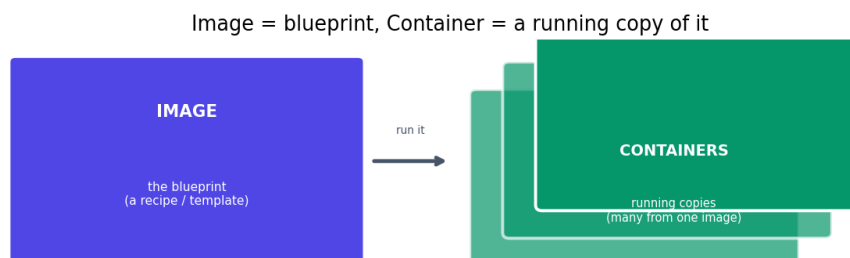


Fig 3 — An image is the blueprint. A container is a running copy of it — you can run many from one image.

The analogy: an image is like a recipe or a class in programming; a container is like the cooked dish or an object made from that class. You build an image once, then you can start many identical containers from it. Share the image, and anyone can run the exact same app.

■ *Quick memory aid: IMAGE = the saved package (sits on disk), CONTAINER = a running instance of that package (alive and working). You build images, you run containers.*

5. The Dockerfile

To make an image, you write a Dockerfile — a simple text file that lists the steps to build your app's environment. It's like a recipe: start from a base, copy your code in, install libraries, and say how to run it.

```
# Dockerfile for a Python ML app

# 1. Start from a base image (Python already installed)
FROM python:3.11-slim

# 2. Set a working folder inside the container
WORKDIR /app

# 3. Copy and install the libraries first (for caching)
COPY requirements.txt .
RUN pip install -r requirements.txt

# 4. Copy the rest of your code in
COPY . .

# 5. Say how to start the app
CMD ['uvicorn', 'main:app', '--host', '0.0.0.0', '--port', '8000']
```

Each line is a step. FROM picks a starting point (here, a slim Python image). WORKDIR sets the folder. COPY brings your files in. RUN installs your libraries. CMD says what to run when the container starts. That's a complete recipe to package your FastAPI ML app.

■ *Copy requirements.txt and install BEFORE copying the rest of your code (as above). Docker caches steps, so this means it only re-installs libraries when they change — making rebuilds much faster. A standard best practice.*

6. The Main Commands

You only need a handful of commands to get started with Docker. Here are the essentials.

Command	What it does
<code>docker build -t myapp .</code>	Build an image from the Dockerfile
<code>docker run myapp</code>	Start a container from the image
<code>docker run -p 8000:8000 myapp</code>	Run and connect a port (to reach the API)
<code>docker ps</code>	List running containers
<code>docker stop <id></code>	Stop a running container
<code>docker images</code>	List your images

The two you'll use most are **build** (make the image from your Dockerfile) and **run** (start a container from it). The -p flag connects a port so you can actually reach your API from outside the container — important for web apps and APIs.

7. Dockerizing an ML App

Let's put it together for your FastAPI Defect Detector. Three small files turn your app into a portable container anyone can run.

- **requirements.txt** — lists your libraries (fastapi, uvicorn, torch, etc.).
- **main.py** — your FastAPI app with the /predict endpoint (from last topic).
- **Dockerfile** — the recipe to package it (shown above).

Then two commands build and run it:

```
# Build the image (once)
docker build -t defect-detector .

# Run a container from it
docker run -p 8000:8000 defect-detector

# Your API is now live at localhost:8000
# ...and this exact container runs the same on any server
```

Now your whole model service — code, libraries, model, Python version — is sealed in one image. You can run it on your laptop, a colleague's machine, or a cloud server, and it behaves identically. This is how your Defect Detector would go to production for real.

■ *Once your app is dockerized, deploying to the cloud (next topic and beyond) becomes much easier — most cloud platforms can run a Docker container directly. Docker is the bridge from your machine to the cloud.*

8. Why It Matters for Jobs

Docker is one of the most valued practical skills for an ML engineer, and a clear gap-filler for your profile. It signals that you can do more than train models — you can ship them like a professional.

Why employers love it	What it shows
Reproducible deployments	You ship reliable software
Industry standard	You know modern practices
Works with cloud	You can deploy to production
Team-friendly	Your work runs on anyone's machine

Putting 'Dockerized my ML API' on a project description is a strong signal. It shows end-to-end thinking: model, API, and deployment. Add a Dockerfile to your Defect Detector and RAG chatbot repos, and you instantly look more production-ready to recruiters.

9. Common Mistakes

Mistake	Why it's a problem	Fix
No requirements.txt	Libraries missing in container	List all dependencies
Copying code before installing	Slow rebuilds	Install deps first, then copy code
Forgetting -p port	Can't reach the API	Map the port with -p
Huge images	Slow to build and share	Use slim base images
Wrong host in CMD	API unreachable	Use --host 0.0.0.0
Putting secrets in the image	Security risk	Use environment variables

Quick Reference Cheatsheet

Concept / Task	Meaning or Command
Docker	packages apps into containers
Container	app + everything it needs, runs anywhere
Image	the blueprint (saved package)
Dockerfile	the recipe to build an image
FROM	start from a base image
WORKDIR	set the working folder
COPY	copy files into the image
RUN	run a command at build time (e.g. pip)
CMD	command to run when container starts
Build image	docker build -t name .
Run container	docker run -p 8000:8000 name
List running	docker ps
Stop	docker stop <id>

The one idea to remember: Docker packages your app with everything it needs into a container that runs identically anywhere. Write a Dockerfile (the recipe), build an image (the package), and run containers (the live copies). It ends 'works on my machine' and is how real ML apps ship.

Next Topic: Kaggle — the world's biggest data science community and platform: free GPUs, datasets, competitions, and notebooks. A perfect place to practise, learn, and build your portfolio.